Calidad del Software

GUION DE PRÁCTICAS 2

Testing Unitario (Google Test) y Generación de Pruebas Asistida por IA

---

1. **Duración estimada:** 3 sesiones (6 horas).
2. **Herramientas:** Visual Studio Code, CMake, Google Test, Google Gemini.

1. INTRODUCCIÓN Y OBJETIVOS

En la práctica anterior establecimos la línea base de calidad. Ahora, antes de poder mejorar (refactorizar) el sistema, necesitamos una red de seguridad que nos asegure que no rompemos nada durante los cambios. Esa red son las **Pruebas Unitarias**.

En esta práctica, nos enfrentaremos a un desafío de arquitectura común: la **Testabilidad**. Incluso si un proyecto tiene clases bien definidas, a menudo la lógica principal se encuentra "atrapada" dentro de la función main(). Como main es el punto de entrada único, no puede ser invocado por un framework de pruebas automatizado.

El Rol de la IA en la Calidad del Software

En esta asignatura utilizaremos Inteligencia Artificial Generativa (**Gemini**), pero no para que "haga la práctica por nosotros", sino para simular un entorno de desarrollo moderno de alto rendimiento.

- **Tu Rol:** Eres el **Ingeniero de Calidad Senior**. Tú decides la estrategia, ejecutas la refactorización arquitectónica, validas los resultados y detectas errores.
- **Rol de la IA:** Actúa como un **Asistente Junior o Herramienta de Productividad**. En esta práctica, la utilizaremos específicamente para la generación de baterías de tests y configuración, una vez que tú hayas preparado la arquitectura.

Los objetivos específicos son:

1. **Refactorizar para la Testabilidad:** Desacoplar manualmente la lógica de negocio del punto de entrada (main) para hacerla verificable.
2. Configurar un entorno de pruebas profesional en C++ utilizando **Google Test** y **CMake**.
3. Aplicar técnicas de **Prompt Engineering** para generar casos de prueba automáticos.
4. Validar y ejecutar tests para asegurar la estabilidad del proyecto.

2. TAREA PREVIA: REVISIÓN Y MEJORA (FEEDBACK P1)

La calidad es un proceso iterativo. Antes de avanzar, revisa la corrección de la Práctica 1:

1. Revisa los comentarios y correcciones indicados por el profesor sobre tu entrega P1.
2. Realiza las modificaciones necesarias para subsanar los errores.

3. CONFIGURACIÓN DEL PROYECTO CON CMAKE

3.1. Estructura y Herramientas

Vamos a crear una nueva carpeta para la práctica 2 (p.e., P2A6, donde A6 sería tu código de grupo) y vamos a generar la siguiente estructura de carpetas y ficheros a partir del contenido recien revisado de la práctica 1:

- **src/:** Mueve aquí **todos** tus archivos de código fuente (.cpp y .h / .hpp).
- **tests/:** Crea esta carpeta y crea dentro el fichero test_main.cpp con este código de inicialización (en el paso 4 lo actualizaremos con código generado por IA):

```C/C++
int main()
{
    return 0;
}
```

- **doc/:** Aquí va tu carpeta html con la documentación generada en la práctica anterior.
- **Doxyfile:** Copia el fichero doxyfile de la práctica anterior en la raíz del proyecto.
- **compile_commands.json: no lo copies** a la carpeta de la práctica 2. En esta práctica será creado automáticamente por CMake.
- **.vscode/:** Copia el contenido de la carpeta .vscode/ de la práctica anterior.
- **.vscode/settings.json:** CMake generará el nuevo fichero compile_commands.json para esta práctica, pero lo hará en una nueva carpeta build/ que creará el propio CMake. Además, CMake busca una herramienta llamada "Ninja". Como nosotros usamos MinGW, debemos decirle explícitamente que use "MinGW Makefiles" o la configuración fallará. Por tanto, actualizamos el contenido del fichero settings.json para reflejar estos dos cambios:

```JSON
{
"sonarlint.pathToCompileCommands":"${workspaceFolder}/build/compile_commands.json",
"cmake.generator": "MinGW Makefiles"
}
```

- **.vscode/c_cpp_properties.json:** este fichero también hace referencia al fichero compile_commands.json, por lo que habrá que hacer una modificación similar:
  - Práctica 1 ⇒ `"${workspaceFolder}/compile_commands.json"`
  - Práctica 2 ⇒ `"${workspaceFolder}/build/compile_commands.json"`

> ⚠️ **Actualización para Doxygen**
>
> Al haber creado una nueva carpeta para esta práctica y movido los archivos fuente a la carpeta src/, tu archivo Doxyfile ha quedado desactualizado. Ábrelo con **Doxywizard**:
>
> 🔨 Actualiza **Project / Source Code Directory** a la nueva carpeta **src**
> 🔨 Actualiza **Project / Destination Directory** a la carpeta **doc**
> ❓ Si **Project / Project Name** hace referencia a **P1**, actualízalo con **P2**
> 💾 **Guarda** el fichero como **P2.Doxyfile**

3.2. Configuración de CMake

Ahora configuraremos **CMake**, una herramienta estándar en la industria para gestionar la compilación de proyectos complejos.

Crea un archivo llamado **CMakeLists.txt** en la raíz de tu carpeta de práctica con este contenido exacto:

```shell
cmake_minimum_required(VERSION 3.14)

# Nombre del proyecto
project(Practica2_Tests CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# --- CORRECCIONES PARA LABORATORIO (WINDOWS/MINGW) ---

# 1. SOLUCIÓN ERROR SSL:
# Desactivamos verificación SSL para evitar fallos de descarga en red universitaria
set(CMAKE_TLS_VERIFY 0)

# 2. SOLUCIÓN CONFLICTO RUNTIME:
# Fuerza a GTest a usar las mismas librerías dinámicas que MinGW
set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)
```

```
# --- CONFIGURACIÓN GOOGLE TEST (Descarga Automática) ---
include(FetchContent)
FetchContent_Declare(
  googletest
  URL https://github.com/google/googletest/archive/refs/tags/v1.14.0.zip
  DOWNLOAD_EXTRACT_TIMESTAMP TRUE
)
FetchContent_MakeAvailable(googletest)

# --- DEFINICIÓN AUTOMÁTICA DE FUENTES ---
# Busca TODOS los .cpp dentro de la carpeta src/
file(GLOB_RECURSE ALL_SOURCES "src/*.cpp")

# IMPORTANTE: Eliminamos 'main.cpp' de esta lista para poder usar
# el resto del código en los tests sin duplicar la función 'main()'
# (Ajusta la ruta si tu main.cpp está en otro lugar)
list(FILTER ALL_SOURCES EXCLUDE REGEX "main.cpp$")

# --- EJECUTABLE PRINCIPAL (La aplicación normal) ---
add_executable(MiPrograma src/main.cpp ${ALL_SOURCES})

# --- EJECUTABLE DE TESTS (Google Test) ---
enable_testing()

add_executable(MisTests
  tests/test_main.cpp  # Tu fichero de tests
  ${ALL_SOURCES}       # Toda la lógica SIN el main.cpp original
)

# Configuración de Includes y Librerías
target_include_directories(MisTests PRIVATE src)
target_link_libraries(MisTests GTest::gtest_main)

# Integración con el explorador de tests de VS Code
include(GoogleTest)
gtest_discover_tests(MisTests)
```

⚠️ **IMPORTANTE: Abre ahora el proyecto desde VS Code**

Para que la configuración de CMake funcione correctamente, no debes abrir solo los ficheros .cpp sueltos. Debes abrir siempre la carpeta raíz del proyecto desde **VS Code** utilizando la opción **File > Open Folder**. Si no lo haces así, la barra de herramientas de CMake no aparecerá.

3.3. Inicialización del Entorno CMake

1. Abre la **Paleta de Comandos** (F1 o `Ctrl+Shift+P`).
2. Empieza a escribir y selecciona: `CMake: Select a Kit`.
3. Selecciona tu compilador: **GCC ... x86_64-w64-mingw32** (o similar).
4. Observa la terminal de "Output" en la parte inferior. CMake comenzará a trabajar:
    - Descargará GoogleTest (puede tardar unos segundos).
    - Configurará el proyecto.
    - Debería terminar con el mensaje: `[cmake] -- Build files have been written to: .../build`.

3.4. Solución de Problemas con CMake

Dado que podemos encontrarnos con distintas situaciones al utilizar **CMake** en **VS Code**, a continuación se muestra una guía orientativa sobre soluciones a adoptar:

Situación	Solución Recomendada	Razón
He cambiado el código C++ y quiero compilarlo.	Pulsar el botón **Build** (o `F7`).	Compila solo lo modificado. Es la vía normal.
He añadido un archivo nuevo o modificado `CMakeLists.txt`.	Guardar (`Ctrl+S`) y pulsar **Build**.	CMake detecta el cambio automáticamente y actualiza el proyecto.
Me da un error raro de CMake, o me he equivocado editando `CMakeLists.txt`.	`Ctrl+Shift+P` (o `F1`) > escribir `CMake: Delete Cache and Reconfigure`	Borra la memoria temporal de CMake y regenera la configuración limpia.
Nada de lo anterior funciona / El proyecto está corrupto.	Borrar la carpeta `build` manualmente y reiniciar VS Code.	Es el "botón de pánico". Obliga a recrear todo el entorno desde cero.

3.5. Refactorización Manual para la Testabilidad

El Problema de Calidad: La función `main()` debe ser lo más simple posible: su única responsabilidad debería ser iniciar el programa y devolver el control al sistema operativo. Si tu main() contiene toda la lógica, no se puede testear automáticamente.

La Solución (Ingeniería Inversa Manual): Como ingeniero de software, debes reestructurar el código sin romper su funcionalidad.

1. **Creación de Archivos**: Crea dos nuevos archivos en la carpeta `src/`: `core_logic.h` y `core_logic.cpp`.
2. **Traslado de Funciones Auxiliares**: Si tienes funciones adicionales en `main.cpp`, muévelas a `core_logic.cpp`.
    - **Importante:** Declara sus cabeceras en `core_logic.h` para que sean visibles por los tests.
3. **Extracción del Main (Función Principal):**
    - Crea una nueva función en `core_logic.cpp` llamada `void run_simulation()`.
    - Mueve todo el código del `main()` (excepto `return 0;`) a esta nueva función.
4. **Gestión de Includes:**
    - Mueve los includes necesarios (`iostream`, `vector`, etc.) a `core_logic.h`.
    - Deja `main.cpp` limpio, incluyendo solo `#include "core_logic.h"`.
5. **Simplificación del Main**:
   Tu archivo `src/main.cpp` debe quedar así:

```
C/C++
#include "core_logic.h"

int main() {
    run_simulation();
    return 0;
}
```

3.6. Blindando el Testing Automatizado

Los sistemas de **testing automatizado** envían datos a gran velocidad. Si tu programa usa **getchar()** (que espera una tecla física) o si "**cin**" falla al recibir un dato inesperado (ej: letra en vez de número), el test se bloqueará en un **bucle infinito**.

1. **Añade esta función** al principio de `src/core_logic.cpp` (antes de `run_simulation`) para gestionar la entrada de forma segura:

```
C/C++
// This function attempts to read an unsigned integer from standard input.
bool safe_read(unsigned int& value) {
    if (std::cin >> value) return true; // Successfully reading

    // Check if EOF was reached
   if (std::cin.eof()) return false; // EOF reached, we can exit the program

    // If we get here, it means input error (e.g., non-numeric input)
    std::cin.clear();           // Clear the error state
    std::cin.ignore(10000, '\n'); // Ignore the rest to prevent infinite loop
    return true;                // Return true to continue the program
}
```

2. Cambia las **instrucciones de entrada** que tengas (del tipo `std::cin >> choice;`) por la siguiente sentencia para lectura segura:

```
C/C++
if (!safe_read(choice)) break; // Exit if EOF is reached
```

3. Si utilizas `getchar();` (o `system("pause")`) al final del bucle, sustitúyela por el siguiente bloque para **pausar de forma segura**:

```
C/C++
std::cout << "Press Enter to continue...";
std::cin.ignore(10000, '\n'); // Clear the input buffer
std::cin.get();               // Wait for the user to press Enter
```

⚠️ **ATENCIÓN: La trampa de mezclar `cin >>` y `getline()`**

En C++, si en tu código pides números u opciones con `cin >>` (o con nuestra función `safe_read`) y luego, más adelante, pides cadenas de texto con `getline(std::cin, texto)`, es muy probable que tu programa se salte la lectura del texto.

¿Por qué ocurre esto? Porque `cin >>` lee el número pero deja el salto de línea (`\n` al pulsar Enter) en el buffer de entrada. Cuando el código llega al `getline()`, este lee ese `\n` residual, cree que el usuario ya ha introducido una línea vacía y continúa la ejecución sin esperar nuevos datos.

Solución: Si vas a usar `getline()` después de haber leído variables simples, asegúrate siempre de limpiar el buffer residual antes usando `std::cin.ignore(10000, '\n');` (de forma similar a como hacemos en la pausa segura), o tus secuencias de tests automatizados fallarán de forma inexplicable.

4. GENERACIÓN DE TESTS ASISTIDA POR IA

Ahora que hemos aislado la lógica en `src/core_logic.cpp`, ya es técnicamente posible testearla. Usaremos la IA para acelerar la creación de la batería de pruebas inicial (*Test Suite*), actuando tú como supervisor de calidad.

4.1. Estrategia: ¿Qué debemos probar?

El objetivo de esta fase es validar la robustez de la función `run_simulation` que acabas de crear. Al ser la función que gestiona el comportamiento principal del programa, es el candidato ideal para detectar fallos graves antes de que ocurran.

4.2. Ingeniería de Prompts para Testing (Caja Blanca)

Vamos a pedirle a Gemini que genere los tests específicos para esa función. Sigue estos pasos exactos:

1. Abre tu fichero `src/core_logic.cpp` y **copia** el código completo de la función `void run_simulation( )`.
2. Ve a Gemini y utiliza el siguiente prompt (copia y completa el hueco):

```shell
Actúa como un experto en C++ y Google Test.
Necesito un archivo de pruebas unitarias para validar la función
'run_simulation'.

CÓDIGO A PROBAR:
[PEGA AQUÍ EL CÓDIGO DE TU FUNCIÓN run_simulation]

REQUISITOS OBLIGATORIOS PARA EL CÓDIGO GENERADO:
1. Genera un archivo completo llamado 'tests/test_main.cpp'.
2. Usa estos includes exactos al principio:
   #include <gtest/gtest.h>
   #include "../src/core_logic.h"
3. Crea al menos 3 casos de prueba para 'run_simulation':
   - Caso Normal (Happy Path): Una ejecución estándar válida.
   - Caso Borde (Edge Case): Entradas vacías, ceros o límites.
   - Caso de Error: Entradas inválidas para verificar que no falla
catastróficamente.
4. NO uses 'using namespace std;'.
5. Si la función usa 'cin' o 'cout', utiliza 'stringstream' para redirigir la
entrada/salida y poder validarla en los tests (ASSERT_THAT o similar).

Genera solo el código C++ listo para copiar.
```

3. Sustituye el contenido del archivo `tests/test_main.cpp` por el código generado.
4. Asegúrate de que todas las modificaciones de los archivos han sido guardadas. Para ello, puedes seleccionar **File / Save All** (o su atajo de teclado ⇒ **Ctrl+K S**).

5. EJECUCIÓN Y DOCUMENTACIÓN DE LOS TESTS

Una vez que tenemos el código generado por la IA, llega el momento de la verdad. En C++, a diferencia de lenguajes interpretados como Python, no basta con guardar el archivo; es obligatorio **compilar** (construir el ejecutable) antes de probar.

5.1. Construcción de los Tests

Seleccionamos el botón Build de la **Barra de Estado,** situada en la zona inferior de la ventana de VS Code.

Resultado esperado: En la terminal inferior debe poner `[100%] Built target MisTests` y `Build finished with exit code 0`:

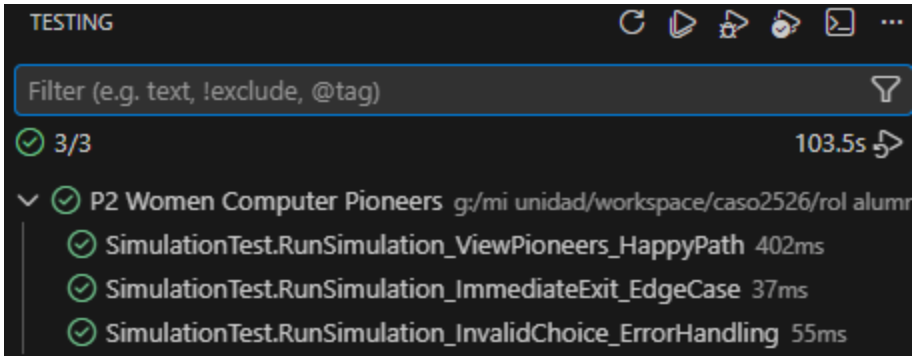
```
[build] [100%] Built target MisTests
[driver] Build completed: 00:02:16.776
[build] Build finished with exit code 0
```

5.2. Ejecución de Tests (Panel Visual)

Si el paso anterior se ha ejecutado correctamente, pasamos a ejecutar los tests. Podemos utilizar el visor gráfico integrado que tiene VS Code:

1. Haz clic en el icono de **Testing** en la barra lateral izquierda.
2. Despliega la lista para ver los grupos de Tests creados (HappyPath, EdgeCase, ErrorHandling).
3. Pulsa el botón de **Play** (Run Tests):

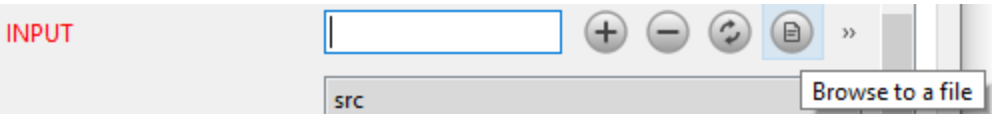
Objetivo: Todos los círculos deben ponerse en **VERDE** (Passed):



5.3. Mantenibilidad: Actualización de la Documentación

En la Práctica 1 aprendimos que la documentación es parte indisoluble de la calidad. Al realizar la refactorización arquitectónica, has creado nuevos elementos que actualmente son "invisibles" para la documentación del proyecto:

1. Añade la **documentación Doxygen** para los nuevos ficheros (`core_logic.cpp`, `core_logic.h` y `test_main.cpp`) y funciones que hayas añadido en esta práctica.
2. Presta especial atención a documentar adecuadamente el contenido de `test_main.cpp`, dado que ahí se encuentra toda la lógica de ejecución de los **tests**, demostrando que entiendes perfectamente qué tipo de pruebas hace cada uno de los **tests generados por IA** en dicho fichero (pruebas *HappyPath*, *EdgeCase* y *ErrorHandling*)
3. Por defecto, **Doxywizard** solo tiene en cuenta los ficheros incluidos en la carpeta **src**, por lo que debemos decirle que también incluya al fichero `test_main.cpp` (ubicado en la carpeta **tests**):
    a. Selecciona la pestaña **Expert** en **Doxywizard**.
    b. Selecciona la variable **Input**.
    c. Pulsa el botón "**Browse to file**" a la derecha de la variable INPUT y selecciona el fichero `tests/test_main.cpp`.



    d. Selecciona la pestaña **Run** y pulsa sobre el botón **Run doxygen** para actualizar la documentación.
    e. Pulsa el botón **Show HTML output** y comprueba que la documentación se ha generado correctamente.
    f. Selecciona **File/Save** (o pulsa **Ctrl+S**) para guardar la configuración actualizada del fichero `P2.Doxyfile`.

6. REQUISITOS DE ENTREGA

Subir al Campus Virtual un archivo **P2X.zip** (ej: P2A5.zip si tu código de grupo es el A5) que contenga:

- **Código Fuente:** Carpeta del proyecto completa **menos** la carpeta `build`.
- **Memoria PDF (P2X.pdf):**
  - **Formato:** Portada, índice, encabezado y pie de página.
  - **Contenido:** Se especifica a continuación.

Contenido de la Memoria (P2X.pdf)

1. **Correcciones P1 (Mejora Continua):** Breve resumen de las mejoras aplicadas al código original tras recibir el feedback de la Práctica 1 (si las hubo).
2. **Refactorización Arquitectónica:** Documenta cómo has transformado el código para hacerlo testable y robusto, prestando especial atención a las modificaciones realizadas en los ficheros `main.cpp`, `core_logic.cpp` y `core_logic.h`. Incluye las capturas de pantalla necesarias para ilustrar dichas modificaciones.
3. **Auditoría de Tests Generados:**
   - Explica detalladamente el objetivo y funcionamiento de cada uno de los tipos de tests generados con IA (ubicados en el fichero `test_main.cpp`).
   - En caso de que alguno de los tests haya fallado inicialmente, explica cómo lo has solucionado.
   - Incluye una captura de pantalla mostrando el resultado exitoso final de las pruebas realizadas (similar a la mostrada en el apartado 5.2).

Fecha límite de entrega

La documentación solicitada deberá ser entregada **antes** de la **finalización** de la **última sesión** habilitada para esta práctica (**6/04/2026**).